

OpenVAF-r Robustness Campaign

Adversarial testing, mutation fuzzing, and the crash/hang paths found and fixed

Ngspice + OpenVAF Enhancements project

July 2026

Contents

OpenVAF-r robustness campaign	2
Summary	5
Method	6
Findings	7
The critical finding: exponential-time nested ?:	8
The hardening: parser depth, include cap, array cap	9
Behaviour-preserving	10
Reproducing	11

OpenVAF-r robustness campaign

A deep robustness audit of the `openvaf-r` Verilog-A → OSDI compiler: the full production-model corpus, ~50 adversarial hand-crafted inputs, and thousands of mutation-fuzzing iterations, run through a classifier that labels every outcome OK / clean-error / panic / segfault / abort / hang. The compiler came back clean on real work, and the campaign surfaced five ways to make it crash or hang on pathological input — all now fixed.

Re-run 2026-07-17. The campaign was replayed against the then-shipped compiler (production corpus + adversarial battery + mutation fuzzer). The memory-safety core was unchanged — 0 panics, 0 segfaults, every standalone model compiled — but the fuzzer found a fifth hang path that the original guards did not cover: an unbounded preprocessor macro-argument collector, plus an unbounded diagnostic-rendering flood behind it. Both are now fixed ([Enhancement-219](#)). Corpus counts below are refreshed to the current `VAF-Models-main` collection.

Round 2 (2026-07-18). A second campaign with a stronger fuzzer — diverse compact-model seeds and new mutation strategies (keyword / attribute / bracket injection) — found that ~5 % of mutated inputs still *crashed* the compiler (exit 101), the E-213 panic class. Triage grouped them into ten distinct root causes (a parser “seems stuck” assert, an empty-span `unimplemented!()`, `TextRange start>end` in span mapping, `unwrap()` on the source-map-back of synthesized exprs, `unwrap_node/unwrap_branch` on wrong-typed builtin args, an include-quote slice, a no-overload candidates `[0]`, and a too-few-arguments args `[0]`), all fixed ([Enhancement-220](#)). A fresh 12 000-iteration fuzz on the fixed compiler is now 0 crashes, 0 hangs. All the panics were already caught by the E-213 hook, so the compiler was never memory-unsafe; the bug was the crash UX and the missing diagnostic.

Round 3 (2026-07-19). A third campaign — the production corpus (again 92 / 92 standalone, identical verdicts) plus ~19,500 mutation-fuzz iterations over diverse compact-model seeds — found three more distinct panic root causes, all the E-213 class (caught by the hook, memory-safe, but a crash instead of a diagnostic): a `begin : block` with a missing name identifier (linked into the item tree as a named scope, then `.name.expect(...)` in `nameres/collect.rs`); a port-flow read of a port that carries both a `Net` and a `Port` decl (an attribute in the port list plus `x = I(<p>)`), which hit `NodeTypeDecl::Port(_) => unreachable!()` in the “expected a port” diagnostic; and an unterminated string literal (a lone `"`) whose quote-stripping slice `src[1..len-1]` became `[1..0]` in `StrLit::value()`. All fixed ([Enhancement-230](#)); a re-fuzz of the fixed compiler is 0 panics, 0 hangs.

Round 4 (2026-07-21). Byte/token mutation of the whole `.va` corpus, grammar-aware structured adversarial inputs, and valid-but-pathological modules that compile through to the backend found three more panics: deeply nested analog operators producing a cached value with no init-time definition, `ddx` on a non-probe unknown, and a malformed module with an empty item list ([E-263](#)). Scaling work in the same round fixed a quadratic module-flatten (a 16 001-element instance array looked like a hang: 2k~1.8 s, 8k~30 s, 16k~100 s, now linear) and a stack overflow ([E-264](#)), and two `laplace` panics on degenerate coefficient lists ([E-265](#)).

The assertion campaign (2026-07-24 ... 07-27). The rounds above ran against a normal release build, where every `debug_assert!` is compiled out. Replaying the corpus under an assertions-enabled build exposed a class the release build silently tolerated: eight crashes across [E-286–E-295](#), [E-307–E-317](#) and [E-324–E-331](#). [E-347](#) closed it: the whole 496-file corpus now compiles assertions-clean. The last one was instructive — the invalid phi operand was minted by `SSAVariableBuilder` during topology linearisation, not by `mir_build` as the earlier diagnosis had assumed.

Round 7 (2026-07-28). A campaign aimed at *wrong code* rather than crashes found nine defects, all fixed: a ddt collapse, two integer-UB SIGTRAPs, IEEE `!/=shift/fast-math` violations, and an OSDI parameter/Jacobian bug ([E-332–E-336](#)).

Round 8 — cross-feature composition (2026-07-29). Every round above MUTATES existing models, and mutants overwhelmingly die at the parser, so the deep stages were barely reached. A generator that instead emits valid-by-construction programs *composing* features developed in isolation (~40 % of inputs reach the backend) found two crashes on legal input, both feature interactions no single-feature test could see: a block merged into ITSELF (`simplify_unconditional_jump_term` with `src == dst`, which is what a case inside a `do-while` folds to), and array parameters never being instance-renamed during flattening, so a module with one could not be instantiated twice ([E-363](#)). A companion complexity sweep over 17 size knobs found no superlinear blowup. One defect from that round was left open at the time — a provably non-terminating analog loop — and is now closed by [E-375](#). It could not stay open: the CFG repair had turned the compiler crash into silent *emission*, and the resulting `.osdi` hung the simulator on its first device evaluation with no diagnostic at all. Rejecting it at compile time also closed three codegen crashes on `disable` used as the only exit from such a loop.

Round 9 — equivalence fuzzing and a system-function sweep (2026-07-30). Two instruments, both aimed at *wrong code* rather than crashes, neither of which needs a reference implementation.

The first is metamorphic: generate a Verilog-A body, rewrite it into a second body that is mathematically identical but structurally different, compile both and require the two `.osdi` models to agree numerically in the same circuit. A difference is a compiler bug on one side, with no judgement about which value is "right". Beyond the curated pairs this runs from a generative grammar over statements — nested `if/else`, bounded `for/while`, named blocks with local declarations, to depth 4–5, with 1–3 composed semantics-preserving rewrites — so control-flow lowering is exercised rather than just expression trees. 160 generated pairs across three seeds, two depths and both one-port and three-terminal bodies: 0 mismatches, with every pair reaching a numeric comparison. Two constraints the compiler enforces correctly had to be learned first, both of which were wasting the budget on compile errors: `$limit` takes a *branch probe*, not an expression, and analog operators are barred from loop bodies and conditional expressions (an operator carries state across time steps, so it must run on every evaluation — there is no hoisting).

The second swept all 94 `$`-prefixed system functions against numeric or textual oracles rather than smoke tests — 24 math functions bit-identical to `libm`, `$vt(400)/$vt` equal to `400/$temperature` to `1.7e-16`, `$abstime` exactly equal to ngspice's time vector, all 16 distributions matching their theoretical moments over 20 000 draws, and a write-then-read round trip over all 17 file-I/O functions. It found three defects, all fixed: [E-376](#) (`$dist_*` returned `real` where the LRM makes it integer — and the signature alone was not enough, since the lowering still emitted a `real` that every integer consumer read as 0), [E-377](#) (a diagnostic with its argument glued to the function name, no newline, a leak, and — with reach far beyond `$simparam` — a `LOG_LVL_MASK` of 8 that collapsed *every* severity to OSDI (`debug`) on `stdout`), and [E-378](#) (a `$fatal` read as non-convergence, so the operating-point solver answered it by

running its whole gmin/source-stepping ladder and then blamed timestep too small).

This is a companion to the [OpenVAF compiler internals](#) guide: that document explains how the compiler works; this one documents how hard it was pushed and what broke.

Summary

Production models compiled	92 / 92 standalone (124 .va files incl. include-fragments) — 0 crashes / hangs / panics
Adversarial hand-crafted inputs	~50 — all rejected cleanly, 0 accepted-invalid
Mutation-fuzzing iterations	tens of thousands — 0 segfaults; panics found + fixed round by round
Cross-feature composition (round 8)	~3 400 valid-by-construction programs; ~40 % reach the backend
Assertions-enabled corpus	496 / 496 compile with zero assertion failures (E-347)
Crash / hang paths found	47 — all fixed. The one long-standing "open by design" entry (a provably non-terminating analog loop) was closed by E-375 , which also closed 3 codegen crashes on <code>disable</code> used as the only exit from such a loop. Rounds 1–3: 18 (E-147 , E-148 , E-219 , E-220 × 10, E-230 × 3). Round 4: 7 (E-263 × 3, E-264 × 2, E-265 × 2). Assertion campaign: 8. Round 7: 9. Round 8: 2 (E-363)

The compiler never crashed on random or garbage input — the only failures were specific, structured pathologies, each turned into a clean bounded diagnostic.

Two lessons generalise beyond this compiler. First, mutation fuzzing plateaus: once the parser is hardened, mutants stop reaching anything new, and the next defects need a generator that produces *valid* programs. Second, panic sites are nondeterministic under rayon — the same input reported `builder.rs:143` or `:690` across runs — so `RAYON_NUM_THREADS=1` is a prerequisite for minimisation, or the reduction chases a moving target.

Method

Every input was compiled with a per-file timeout under `RUST_BACKTRACE=1`, and the result classified from the exit signal and output:

- OK — exit 0 (a `.osdi` was produced);
- clean error — nonzero exit with a diagnostic, no panic/crash;
- panic — a Rust panicked at ... (an internal compiler error);
- segfault / abort — killed by SIGSEGV / SIGABRT (e.g. a stack overflow);
- hang — did not finish within the timeout.

Only panic / segfault / abort / hang count as robustness bugs; a clean error on malformed input is the desired outcome.

Phase A — the real production corpus

Every model in the compact-model test suites (BSIM3/4/6, BSIM-CMG/IMG/SOI/BULK, PSP 102/103/104, HICUM L0/L2, MEXTRAM, VBIC, EKV/EKV3, HiSIM, ASMHMT, r2/r3_cmc, diode_cmc, ...) was compiled. The reproducible driver is `VA_TEST/compile_all.py` over the `VA-Models-main` collection — 124 `.va` files, of which 92 are standalone models and 32 are include-fragments.

Result: 92 / 92 standalone models compiled with 0 crashes / hangs / panics. The 32 include-fragment files — function libraries and macro-body pieces meant to be ``included`, not compiled standalone — report a clean error, as expected (17 of them; the other 15 happen to compile anyway).

Phase B — adversarial hand-crafted inputs

~50 inputs targeting the parser, preprocessor, and lowering: unterminated constructs (block comment, string, module, parenthesis), malformed numbers (`1e999999`, `4'b2`, `999999999'b1`, `1.2.3`), macro bombs (``define A A`, mutual recursion, an exponential expansion chain), circular / missing includes, duplicate declarations, unicode identifiers and embedded null bytes, and extreme nesting / sizes.

Result: all rejected cleanly; 0 accepted-invalid inputs. Five structured pathologies produced a crash or hang (below); everything else was a clean error. Macro recursion is already capped; ``include` recursion was not.

Phase C — mutation fuzzing

4,000 iterations, each taking a real model and applying a random mutation — byte flips, truncations, run duplication, delimiter injection, deep-nesting injection — then compiling it.

Result: 0 panics, 0 segfaults across 4,000 iterations. The compiler never crashed on mutated input; a few mutations produced hangs, all instances of the structured pathologies below (macro-expansion / nesting blow-ups).

Findings

Severity	Trigger	Symptom (before)	Status
critical	nested ?: chain, depth ≥ 30	$O(2^n)$ validation $\rightarrow$ compiler hangs; reachable via macros	fixed (E-147)
low	expression nested ~4k–32k deep (unary / binary / parens / calls)	recursive-descent stack overflow (SIGABRT)	fixed (E-148)
low	file that <code>`includes</code> itself	uncapped include recursion $\rightarrow$ stack overflow	fixed (E-148)
low	absurd array range <code>real x[0:100000000]</code>	expanded element-by-element $\rightarrow$ memory exhaustion / hang	fixed (E-148)
high	<code>`name(</code> with a stray directive ( <code>`include</code> , <code>`ifdef</code> , ...) in its args	preprocessor macro-arg collector never advances $\rightarrow$ infinite loop (hang)	fixed (E-219)
low	thousands of parse errors (deeply nested garbage)	every diagnostic rendered with source context $\rightarrow$ ~40 s	fixed (E-219, render cap 128)
low	<code>begin :</code> block with a missing name identifier	linked as a named item-tree scope $\rightarrow$ <code>.name.expect()</code> panic	fixed (E-230)
low	port-flow read <code>I(<p>)</code> of a port-list-attributed port	”expected a port” report hit <code>Port(_)</code> $\Rightarrow$ <code>unreachable!()</code> panic	fixed (E-230)
low	unterminated string literal (a lone <code>"</code> , e.g. in an attribute)	quote-strip slice <code>[1..len-1]</code> $\rightarrow$ <code>[1..0]</code> panic	fixed (E-230)

Round 2 (E-220) added ten more panic root causes and Round 3 (E-230) three; see those write-ups for the full list. Every one was a compiler panic (caught by the E-213 hook, never memory-unsafe) turned into a clean diagnostic.

The low-severity findings all require input a human or real model would never write. The others are worse: ~30 nested conditionals (E-147) is easily produced by macros in a real model, and the E-219 macro-argument loop is trivially reached by injecting (near any ``include`/macro token — both *hang* the compiler, the worst failure mode. E-219 was found only by re-running the campaign against the shipped binary (see the 2026-07-17 note above); its two paths sit in the preprocessor and the diagnostic sink, which the E-147/E-148 parser/validator guards did not touch. See [Enhancement-219](#) for root cause and fix.

The critical finding: exponential-time nested ?:

Root cause

The body validator visits every expression in `validate_expr`, whose match ends by recursing into the children:

```
_ => ()  
}  
self.parent.body.exprs[expr].walk_child_exprs(|child| self.validate_expr(child))
```

Arms that fully validate their own operands return early to skip that generic walk — the `Call` and `Path` arms both do. The **Select** (ternary) arm did not: it validated `cond`, `then_val` and `else_val` via `validate_condition`, then *fell through* to `walk_child_exprs`, which validated `then_val` and `else_val` a second time. Each ternary therefore validated its branches twice; for a chain of N nested ?: this compounds to 2^n validations.

Profiling a hanging compile with a live stack sample confirmed it exactly — an unbounded `validate_expr` $\rightarrow$ `validate_condition` $\rightarrow$ `validate_expr` $\rightarrow$... recursion with a doubling call count.

The fix

Add a `return`; at the end of the `Select` arm, matching the `Call` / `Path` arms. The arm already validates all three children, so the generic walk was pure redundant work — removing it is behaviour-preserving and turns validation from $O(2^n)$ into $O(N)$:

nesting depth	before	after
40	hang (> 30 s)	0.09 s
160	2^{160} (hopeless)	0.11 s
2000	—	1.6 s (linear)

Full write-up: [Enhancement-147](#).

The hardening: parser depth, include cap, array cap

The three lower-severity findings are pathological-input crashes — a robust compiler should reject them cleanly rather than die. [Enhancement-148](#) adds bounded limits.

Parser expression-depth limit. The Pratt parser recurses through `atom_expr` for every level of nesting *and* builds a left-leaning tree for operator chains — so both an over-deep recursion and an over-deep tree (which later passes `recurse` over) could overflow the stack. A shared `expr_depth` counter — incremented on each `atom_expr` (recursion) and per operator in the `expr_bp` loop (chain length) — bounds the total expression-tree depth at 1000 and recovers to the next statement boundary.

**`include** recursion cap. An `include_depth` counter caps nesting at 64 and emits a new `IncludeRecursionLimit` diagnostic — a self-including file is reported instead of overflowing the stack, mirroring the existing macro-recursion guard.

Array element cap. An array-shaped declaration is flattened into one scalar element per index. A shared `array_elem_count` helper caps the expansion at $2^{20} \approx 1.05$ M elements and emits a new `ArrayTooLarge` diagnostic, applied at every expansion site: variable arrays, parameter arrays, net/port buses and array function returns (item-tree lowering), and instance arrays — in *both* the item-tree lowering and the elaboration pass, which re-expands `dev s[0:N]()` into rendered text.

Behaviour-preserving

Both fixes are correctness/robustness changes with no effect on valid programs, and that was checked rather than assumed:

- Corpus head-to-head. Every one of the 92 standalone production models compiles to the identical verdict before and after each fix — 0 flips — and BSIM4 (12.6 k lines) still compiles in ~2.3 s. (E-219 was checked the same way: the whole corpus is byte-for-byte unchanged across it.)
- Unit tests. The parser, preprocessor, `hir_def`, `hir_ty` and `sim_back` suites pass with no snapshot changes; the build is warning-free.
- Targeted examples. `examples/nested_cond_examples/` (7/7, the exponential fix) and `examples/robustness_examples/` (17/17, the hardening) pin the behaviour: every pathological input now errors cleanly in well under a second, and valid deep-but-reasonable input (nested ternary depth 30, parentheses depth 100, a 100-term sum, small variable and instance arrays) still compiles.

Reproducing

The campaign is a small Python harness: a classifier that runs `openvaf-r` on an input under a timeout and labels the outcome, plus three drivers — compile the real corpus (Phase A), a battery of hand-crafted adversarial files (Phase B), and a mutation fuzzer that perturbs real models (Phase C). Localizing a hang used two tools: the compiler's own `--dump-j son / --dump-unopt-mir` flags to bisect the pipeline phase, and macOS `sample <pid>` on the live process to name the exact hot function.